# CSE 5104 Project 1
# Concrete Compressive Strength Analysis
Part 3: Technical Appendix

Yuan Fan Ezra Lin Rong Bao

## Algorithmic Design & Implementation Details

We followed the supplied Part 1 R Markdown implementation and reran its experiment blocks on the provided Excel file. The training set contains 900 rows, and the testing set contains the 130 rows at positions 501 through 630, using R's one-based indexing. The split is made before computing transformations. All eight predictors are retained, the response remains in MPa, and an intercept is included in every model. The supplied dataset contains 1,030 rows and nine numeric columns; its original row order is preserved.

Let $X \in \mathbb{R}^{n \times p}$ contain the predictors and let $A = [\mathbf{1}\ X]$ include the intercept column. For parameters $\beta \in \mathbb{R}^{p+1}$, our objective is

$$J(\beta) = \frac{1}{n}\sum_{i=1}^{n}(\mathbf{a}_i^\top \beta - y_i)^2 = \frac{1}{n}\|A\beta - \mathbf{y}\|_2^2.$$

Differentiating the squared residual for each observation gives $2\mathbf{a}_i(\mathbf{a}_i^\top \beta - y_i)$. Summing and dividing by $n$ produces

$$\nabla J(\beta) = \frac{2}{n}A^\top(A\beta - \mathbf{y}), \qquad \beta^{(t+1)} = \beta^{(t)} - \alpha \nabla J(\beta^{(t)}).$$

This is full-batch gradient descent, so each update uses all training observations. The intercept starts at zero along with every slope. There is no penalty term, momentum, adaptive rate, or random initialization. The same function handles a single predictor by accepting a one-column matrix.

The following pseudocode matches the original routine, including its loss indexing and stopping behavior.

```
function gradient_descent(X, y, alpha, max_iter):
    A = prepend_column_of_ones(X)
    beta = zeros(number_of_columns(A))
    losses = empty_list()
    repeat max_iter times:
        error = A * beta - y
        append mean(error^2) to losses
        gradient = (2 / number_of_rows(A)) * transpose(A) * error
        beta = beta - alpha * gradient
        if any beta is not finite: break
```

```
if all beta are finite:
    append mean((A * beta - y)^2) to losses
return beta, losses
```

The function stops after its fixed iteration budget or when a coefficient becomes non-finite. It does not stop when the loss is small or when the gradient reaches a tolerance. Consequently, a completed finite run is not necessarily converged. Loss index one describes the zero-parameter model before any updates; for successful runs, the appended value describes the final parameters. In the figures, we relabel these positions as update counts starting at zero.

Table 1 gives the multivariate settings and final training results. The univariate experiments use $\alpha = 10^{-6}$ except for Age, which uses $10^{-5}$, with a maximum of 1,000,000 updates each. These are the existing Part 1 settings rather than rates selected using testing performance. We report unsuccessful runs explicitly instead of treating them as valid fitted models.

**Table 1:** Multivariate gradient descent settings and final training scores. All three runs remain finite; first positive indicates the first update with $R^2 > 0$.

| Inputs | $\alpha$ | Updates | MSE | $R^2$ | First positive |
|---|---|---|---|---|---|
| Raw | $1.00 \times 10^{-8}$ | 100,000 | 107.501 | 0.6119 | 136 |
| Standardized | 0.0100 | 10,000 | 104.144 | 0.6240 | 56 |
| Log | $1.00 \times 10^{-5}$ | 50,000 | 130.864 | 0.5276 | 557 |

For each dataset, we calculate

$$\mathrm{MSE} = \frac{1}{n}\sum_i (y_i - \hat{y}_i)^2, \qquad R^2 = 1 - \frac{\mathrm{MSE}}{n^{-1}\sum_i (y_i - \bar{y})^2}.$$

The denominator uses the mean of the dataset being evaluated and divisor $n$, matching the supplied code. R's `var()` instead uses $n-1$, so substituting it directly would slightly change the result. MSE has units of MPa squared, while its square root is in MPa. A negative $R^2$ means the predictions are worse than predicting that evaluation set's mean.

## Feature Scaling & Gradient Descent Dynamics

We standardize each predictor using its training mean $\mu_j$ and training sample standard deviation $s_j$:

$$z_{ij} = \frac{x_{ij} - \mu_j}{s_j}.$$

The same stored values transform the testing observations; we do not recompute them on the testing set. The response is not standardized. Figure 1 compares the original and standardized training losses, using a logarithmic iteration axis so that both early progress and long runs are visible. Both runs achieve positive training $R^2$, but their final losses differ because they have not reached exactly the same point in parameter space.

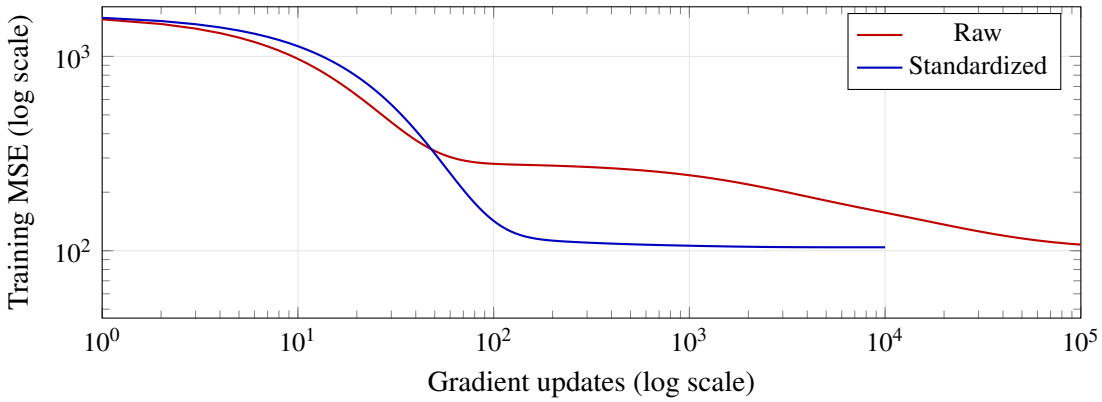



**Figure 1:** Original and standardized training losses at logarithmically spaced update counts. The zero-update loss is omitted because a logarithmic axis cannot display zero.

The original-feature run ends at MSE 107.501 and $R^2 = 0.6119$ after 100,000 updates. The standardized run ends at MSE 104.144 and $R^2 = 0.6240$ after 10,000 updates, essentially matching the least-squares benchmark. In this experiment, standardization therefore reaches a better fit with one tenth of the update budget. Since the learning rates also differ, this is a comparison of the specified training configurations, not a controlled claim that scaling alone always yields a tenfold speedup.

The reason is visible in the curvature of the objective. Its Hessian is $H = 2A^\top A/n$, and the largest eigenvalue limits a stable constant learning rate. Large original units, particularly the aggregate quantities, create steep directions that force very small updates. Correlated columns and the uncentered intercept can still leave other directions progressing slowly. Standardization makes the scales more comparable and centers the predictors, although it does not remove predictor correlation.

For the optimal-solution comparison, we use `lm()` instead of our gradient descent routine. Original and standardized predictors produce the same training MSE, 104.14374, and testing MSE, 141.31386. If $\gamma_j$ is a standardized slope, then $\beta_j = \gamma_j/s_j$ and $\beta_0 = \gamma_0 - \sum_j \gamma_j \mu_j/s_j$. Thus the transformation changes the coordinates of the coefficients while preserving the available fitted values. This equivalence relies on including an intercept, nonzero scaling factors, and no regularization.

## Linear vs. Non-Linear Feature Transformations

For the nonlinear experiment, every predictor is replaced by $\log(1+x_j)$ using R's `log1p()`. Adding one allows zero quantities of slag, fly ash, or superplasticizer without taking the logarithm of zero. The transformation is applied to the predictors only, not to strength. We use the same numerical units throughout, because changing units before adding one would change this transformation.

Figure 2 adds the logarithmic run to the loss comparison. Logarithms compress large values, but this alone does not center the columns or eliminate correlation. With the supplied rate of $10^{-5}$ and 50,000 updates, the final logarithmic GD loss must be distinguished from the optimized logarithmic model reported below. The fixed-budget gradient descent experiments show numerical behavior; the `lm()` fits establish the best least-squares solutions for these three feature representations. The logarithmic GD run finishes at MSE 130.864

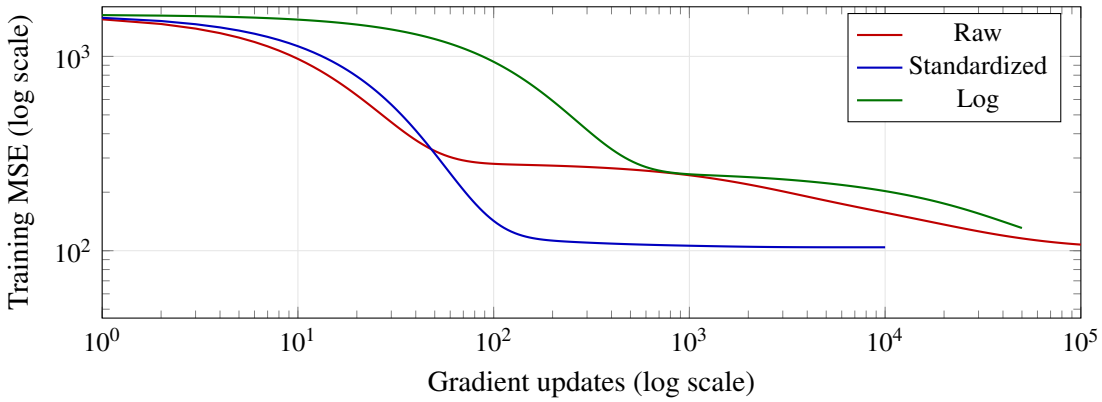



**Figure 2:** Training losses for all three gradient descent experiments. Both axes use logarithmic scales; curves end at their actual iteration budgets.

and $R^2 = 0.5276$, well above its least-squares loss of 55.832. It remains finite but is not close to the optimum. Its first positive $R^2$ occurs at update 557, compared with 136 for raw inputs and 56 for standardized inputs. Thus this particular log-transformed run does not outperform the standardized run in convergence speed, even though its feature representation supports a substantially better optimal fit.

Table 2 gives all four required metrics for the optimized models. The logarithmic representation reduces testing MSE from 141.314 to 54.385, approximately a 61.5% reduction. Testing $R^2$ increases from 0.3685 to 0.7570. The smaller training-to-testing gap is useful evidence of better performance on this particular split, but the fixed contiguous holdout may differ systematically from the training rows. A separate local evaluation would be needed before claiming broad generalization.

**Table 2:** Optimal least-squares model scores using the prescribed 900/130 split. MSE is measured in $\text{MPa}^2$.

| Model | Train MSE | Test MSE | Train $R^2$ | Test $R^2$ |
|---|---|---|---|---|
| Raw | 104.1437 | 141.3139 | 0.6240 | 0.3685 |
| Standardized | 104.1437 | 141.3139 | 0.6240 | 0.3685 |
| Log | 55.8324 | 54.3849 | 0.7984 | 0.7570 |

Unlike standardization, logarithms change the functions available to the model. An original-feature model has constant partial slope $\beta_j$, while a logarithmic model has partial slope $\theta_j/(1+x_j)$. For example, the fitted Age coefficient is 9.03451, giving a change of $9.03451 \log[(1+a_2)/(1+a_1)]$ between two ages. This permits diminishing increments with increasing age. It does not introduce explicit ingredient interactions, and the observed improvement should not be attributed to a particular physical mechanism without further analysis.

## Univariate vs. Multivariate Model Dynamics

Table 3 compares the eight univariate gradient descent runs with their least-squares benchmarks. We judge training difficulty using the stated rates, achieved losses, and numerical failures, rather than using coefficient significance as a substitute for optimization behavior. The million-update budget is substantial, but raw units

can still make learning slow. In a one-predictor model, the slope and intercept can be strongly coupled when the predictor has a large mean relative to its spread.

**Table 3:** Univariate gradient descent results after 1,000,000 updates. All runs remained finite. A dash means positive $R^2$ was not reached.

| Predictor | GD MSE | GD $R^2$ | OLS $R^2$ | First positive |
|---|---|---|---|---|
| Cement | 219.784 | 0.2066 | 0.2644 | 9 |
| Slag | 342.699 | -0.2371 | 0.0254 | – |
| Fly ash | 361.416 | -0.3047 | 0.0425 | – |
| Water | 333.601 | -0.2043 | 0.0767 | – |
| Superplasticizer | 321.337 | -0.1600 | 0.1018 | – |
| Coarse aggregate | 297.344 | -0.0734 | 0.0163 | – |
| Fine aggregate | 309.703 | -0.1180 | 0.0226 | – |
| Age | 242.970 | 0.1229 | 0.1229 | 116941 |

Cement is the easiest run under these settings: it first reaches positive $R^2$ after nine updates and ends at 0.2066. Age first reaches positive $R^2$ at update 116,941 and ends at 0.1229. The other six runs remain finite but have negative final $R^2$, so they are harder to train with these settings. Nevertheless, every optimized univariate model has positive $R^2$. This confirms that the negative GD results reflect incomplete optimization rather than an inability of those one-predictor models to outperform the mean. Exactly two GD models satisfy the positive-$R^2$ target.

Table 4 compares optimal univariate slopes with the corresponding original multivariate slopes. The univariate values come from eight separate `lm()` fits, not from unconverged GD estimates. Several signs change: fly ash and both aggregate variables have negative univariate slopes but positive multivariate slopes. Cement remains positive, although its estimated slope also changes. These differences show that an isolated ingredient relationship is not the same as its relationship after accounting for the other recorded inputs.

**Table 4:** Optimal original-scale slopes from separate univariate regressions and the eight-predictor regression. Slopes are in MPa per predictor unit.

| Predictor | Univariate slope | Multivariate slope |
|---|---|---|
| Cement | 0.080784 | 0.132449 |
| Slag | 0.031598 | 0.124728 |
| Fly ash | -0.053404 | 0.106147 |
| Water | -0.207016 | -0.134100 |
| Superplasticizer | 0.888141 | 0.117239 |
| Coarse aggregate | -0.027382 | 0.028547 |
| Fine aggregate | -0.030582 | 0.033835 |
| Age | 0.095208 | 0.118804 |

With an intercept, the univariate slope for predictor $j$ is $\mathrm{Cov}(X_j, Y)/\mathrm{Var}(X_j)$. Under the fitted full model, orthogonality of least-squares residuals yields the sample identity

$$b_j^{\mathrm{uni}} = \beta_j + \sum_{k \neq j} \beta_k \frac{\mathrm{Cov}(X_j, X_k)}{\mathrm{Var}(X_j)}.$$

The omitted predictors contribute whenever they correlate with the included predictor. For example, cement and fly ash have training correlation about $-0.480$, while water and superplasticizer have correlation about $-0.658$. A univariate fly ash slope can therefore reflect the lower cement quantities that accompany fly ash. This is the omitted-variable explanation for coefficient differences; it does not establish causal effects for the full model either.

## Statistical Inference & Regression Analysis

We obtain two-sided coefficient $t$-test p-values from `summary(lm(...))`. Table 5 consolidates the eight predictor results; the intercept is omitted because the question concerns predictor significance. The intercept's interpretation also changes after centering. The tests evaluate a zero coefficient conditional on the other included predictors and the specified functional form, so they should not be read as a ranking of practical usefulness.

**Table 5:** Two-sided OLS slope p-values in the three feature configurations. Values are rounded; no multiple-testing correction is applied.

| Predictor | Original | Standardized | Logarithmic |
|---|---|---|---|
| Cement | $2.49 \times 10^{-40}$ | $2.49 \times 10^{-40}$ | $1.04 \times 10^{-85}$ |
| Slag | $6.43 \times 10^{-26}$ | $6.43 \times 10^{-26}$ | $6.96 \times 10^{-36}$ |
| Fly ash | $3.30 \times 10^{-13}$ | $3.30 \times 10^{-13}$ | 0.2919 |
| Water | 0.0018 | 0.0018 | $1.69 \times 10^{-20}$ |
| Superplasticizer | 0.2431 | 0.2431 | $8.44 \times 10^{-6}$ |
| Coarse aggregate | 0.0056 | 0.0056 | 0.4049 |
| Fine aggregate | 0.0030 | 0.0030 | 0.0892 |
| Age | $8.61 \times 10^{-74}$ | $8.61 \times 10^{-74}$ | $1.75 \times 10^{-192}$ |

At the conventional 0.05 threshold, all original-model predictors except superplasticizer are significant. Standardization leaves those slope p-values unchanged to numerical precision, since each slope and its standard error are rescaled by the same factor. Logarithmic transformation changes the pattern: cement, slag, water, superplasticizer, and age are significant, while fly ash and both aggregates are not. This happens because logarithms alter the fitted relationships and residual variation, not merely the units used to express an identical model.

The significance results do not map directly onto easy or hard gradient descent runs. Age is strongly significant and trains successfully, but a useful predictor can still be numerically difficult when represented in poorly scaled units. Conversely, a positive univariate $R^2$ does not guarantee a significant conditional coefficient in a multivariate fit. Statistical signal and optimization conditioning answer different questions, which is why we report both sets of evidence.

These are conventional OLS p-values rather than robust or cluster-adjusted tests. Their usual interpretation assumes an adequate mean model and independent errors with constant variance; exact finite-sample inference additionally relies on normal errors. Related mixtures or measurements at several ages could weaken the independence assumption, but group identifiers are not supplied. We have not established these assumptions or applied a multiple-testing correction, so the p-values are descriptive evidence for this assignment

rather than proof that an ingredient causes a strength change.

## Code & Reproducibility Access

All analysis code is provided in a public GitHub repository, consolidated into the single file `Project 1 Section 1.Rmd`. The repository also includes the Excel dataset, its original attribution, a README, and the exported results. Repository link: `https://github.com/Rome2014/CSE-5104-Project-1`.

The verified analysis environment is R 4.6.1 with `readxl` 1.5.0.1 and `knitr` 1.51. The remaining analysis functions use standard R packages, including `stats`, `graphics`, `grDevices`, and `utils`. Rendering the R Markdown as a Word document additionally requires `rmarkdown` and Pandoc; document rendering is not required to execute the analysis chunks. The exported `sessionInfo.txt` records the actual analysis environment.

Download or clone the repository, then set the R working directory to its root. Opening the R Markdown file alone does not necessarily change the working directory of an existing R session. For example, replacing the first path below with the local repository location:

```
setwd("/path/to/concrete_strength_analysis")
getwd()
# Install these packages if they are not already available:
install.packages(c("readxl", "knitr"))
```

Keep `Concrete_Data.xls` in the repository's `concrete+compressive+strength` directory, without sorting or changing its rows. Run the setup chunk, then Sections 1.1 through 1.7 in order, waiting for each chunk to finish. Skip the unrelated `cars` and `pressure` tutorial chunks. Section 1.1 reads the data and creates the output directory; Section 1.2 defines the optimizer; Sections 1.3–1.7 run and export the experiments. The univariate runs may take substantial time because each permits one million updates. If restarting earlier experiments after Section 1.7, rerun from data preparation because that section reassigns shared variables.

The relative output directory is `projects_results`. The code exports model metrics, scaling parameters, coefficient estimates, p-values, multivariate loss histories, predictor correlations, and univariate diagnostics as CSV files, along with two PNG convergence figures and the session information. Rerunning overwrites matching output files. The included `Run_Plots.pdf` was captured by the execution session's graphics device; the R Markdown itself exports the named PNG figures. No random seed is needed because neither the split nor the optimizer is stochastic. Tables in this report use rounded display values, and the embedded curves use sampled update counts from the exported loss histories; the CSV files retain greater precision for checking results.

## AI Disclosure

We used ChatGPT (OpenAI) to help us understand coding approaches, troubleshoot errors, and clarify statistical concepts. We also used it to assist with drafting and revising parts of the report, explaining results,

improving organization and language, and formatting the document in LaTeX. The project team reviewed the analysis and conclusions.